Listing 4.9 OpenTelemetry collector configuration, receivers

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: "0.0.0.0:4317"
  prometheus:
    config:
      scrape_configs:
       - job_name: "holly"
         static_configs:
         - targets: ["holly:8765"]
```

Enables and configures the OTLP over gRPC receiver (http://mng.bz/4Drj) we're using for ingesting traces. We're declaring here that it should listen on port 4317, the official, IANA-registered port for this scheme and protocol.

Here, we enable the Prometheus receiver (http://mng.bz/QP9v) and configure it so that it scrapes our sample app.

Note that Docker Compose makes it so that the `holly` app will be reachable under said name. Let's move on to the egress path, the exporters.

Listing 4.10 OpenTelemetry collector configuration, exporters

```
exporters:
  logging:
    loglevel: debug
  prometheus:
    endpoint: "0.0.0.0:9292"
  jaeger:
    endpoint: "jaeger:14250"
    tls:
      insecure: true
```

We enable and configure the Prometheus exporter (http://mng.bz/XNla), turning our collector, effectively, in a pass-through.

Next up is the Jaeger gRPC Exporter (http://mng.bz/yQjd), which, as the name suggests, allows us to export (trace) data via gRPC to a Jaeger backend destination.

One thing before we move on: remember that listing 4.8, which showed the Jaeger configuration in Docker Compose, pointed out port 14250? That's exactly what we plug in here, in listing 4.10, so that the OpenTelemetry collector knows where to send the traces.

For the sake of keeping things simple in this example, we're using the Prometheus exporter to allow scraping by a Prometheus server. In a production setup, where federation and long-term storage are relevant, you likely will want to instead use the Prometheus Remote Write Exporter (http://mng.bz/MBGB) here. This enables you to ingest metrics into backends, such as Cortex, Thanos, or managed-OSS services, like Amazon Managed Service for Prometheus (https://aws.amazon.com/prometheus/).

> **TIP** In the collector config, components (receivers, processors, exporters, connectors, and pipelines) are defined by a component identifier in the `type[/name]` format. You can define components of a given type more than once as long as the identifiers are unique. For example, you could have two metrics pipelines, one called `metrics/dev` and one `metrics/prod`, with different backends.

Finally, we get to use all these receivers and exporters to define our pipelines, as shown in the following listing.